Minimum Waiting Time

AlgoExpert · Easy · Greedy Algorithms

The problem

- Minimum Waiting

Intuition

- Short queries should run first, because each query's duration is waited on by everyone after it.

Brute force vs optimal

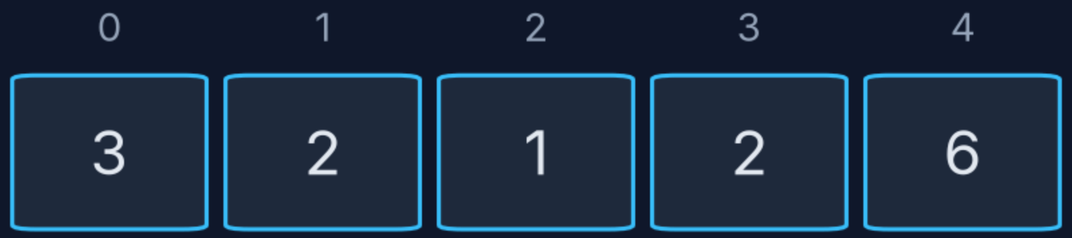
- Brute force: try every order → n! permutations
- Greedy: shortest first is optimal → O(n log n)
- Each duration counts once per query behind it

Approach

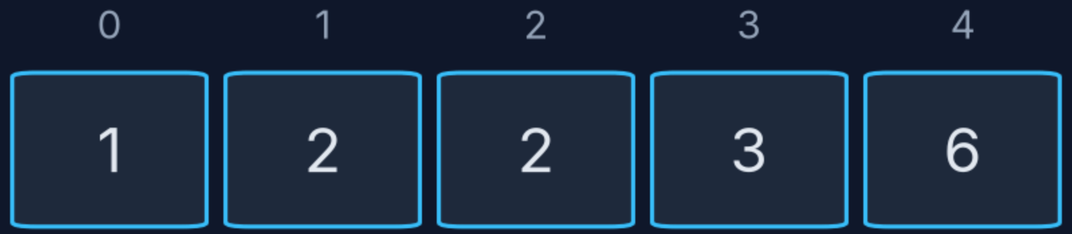
- 1. Sort durations ascending
- 2. Each duration is waited on by the queries remaining after it
- 3. Add duration times queries left
- 4. Return the total

Worked example

queries

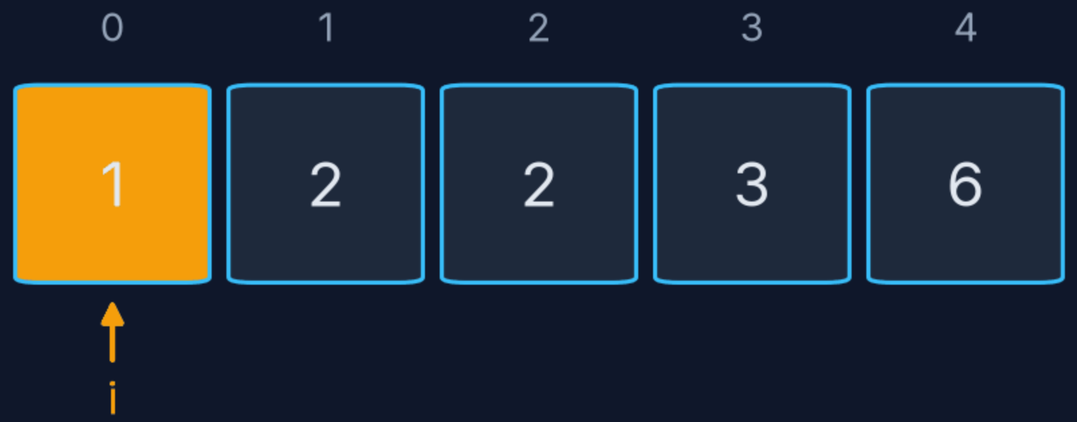


sorted



Sort: [3, 2, 1, 2, 6] → [1, 2, 2, 3, 6]

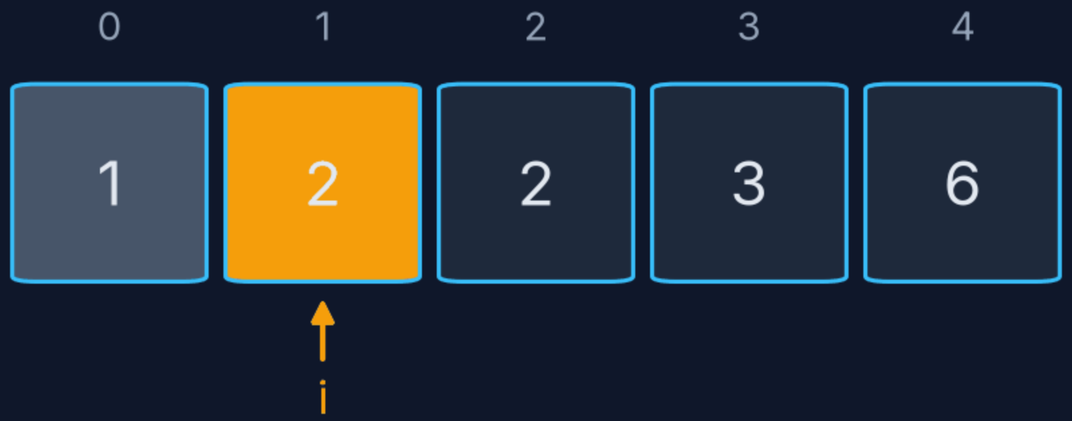
Worked example




```
duration 1 × 4 waiting = 4
total = 4
```

1 s is waited on by 4 queries → total 4

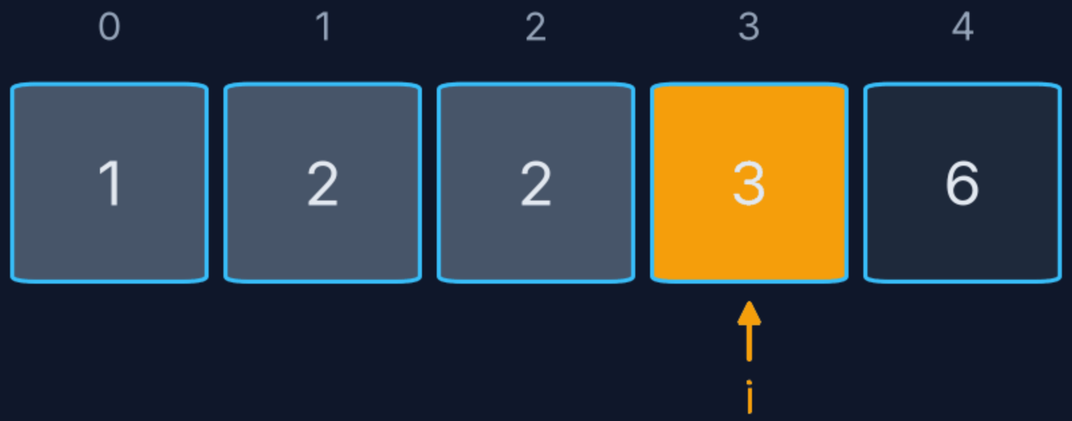
Worked example




```
duration 2 × 3 waiting = 6
total = 10
```

2 s is waited on by 3 queries → total 10

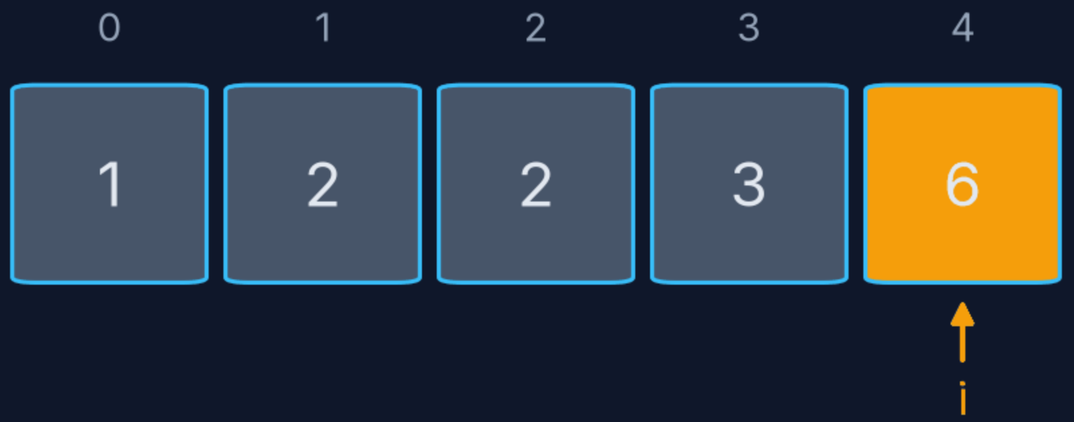
Worked example




```
duration 3 × 1 waiting = 3
total = 17
```

3 s is waited on by 1 queries → total 17

Worked example




```
duration 6 × 0 waiting = 0
total = 17
```

6 s is waited on by 0 queries → total 17

Java solution

```java
public class MinimumWaitingTime {
    public static int minimumWaitingTime(int[] queries) {
        Arrays.sort(queries);
        int total = 0;
        for (int i = 0; i < queries.length; i++) total += queries[i] * (queries.length - 1 - i);
        return total;
    }
}
```

Complexity

- Time: queries with durations run one at a time; choose an order minimising the total time all queries spend waiting. Shortest-first is optimal. Time: O(n log n)
- Space: O(1)

Edge cases & pitfalls

- The last query adds 0 (nobody waits for it)
- Sorting in place mutates input (mention it)
- Use long for large totals

Interview tips & follow-ups

- Prove with an exchange argument
- Follow-up: weighted queries (Smith's rule: sort by time/weight)

Recap

- Short queries should run first, because each query's duration is waited on by everyone after it.
- Complexity: queries with durations run one at a time; choose an order minimising the total time all queries spend waiting. Shortest-first is optimal. Time: O(n log n)